

## 第 12, 13 回 高速フーリエ変換

執筆: 安永憲司

分割統治法により、整数の掛け算と行列の掛け算を高速化することができた。次の対象は**多項式** (*polynomial*) である。次数高々  $d$  の多項式同士の積は次数高々  $2d$  の多項式である。多項式  $A(x) = a_0 + a_1x + \cdots + a_dx^d$  と  $B(x) = b_0 + b_1x + \cdots + b_dx^d$  の積を

$$C(x) = A(x) \cdot B(x) = c_0 + c_1x + c_2x^2 + \cdots + c_{2d}x^{2d}$$

とすると、各係数  $c_k$  ( $k = 0, \dots, 2d$ ) について

$$c_k = a_0b_k + a_1b_{k-1} + \cdots + a_kb_0 = \sum_{i=0}^k a_ib_{k-i}$$

と書ける。ただし、 $i = d+1, \dots, 2d$  については  $a_i = b_i = 0$  と考える。この式に従うと、 $c_k$  の計算は  $O(k)$  回の演算ででき、係数同士の演算の計算量を  $O(1)$  としたとき、 $c_0, c_1, \dots, c_{2d}$  を求める計算量は  $O(d^2)$  である。ここではこの計算を高速化する方法（高速フーリエ変換 (fast Fourier transform, FFT)）を見てみる。この手法自体は、信号処理 (signal processing) 分野で発展したものである。

**例 0.1.**  $A(x) = 4x^3 + 3x^2 + 2x + 1$  と  $B(x) = 8x^3 + 7x^2 + 6x + 5$  の積は、

$$\begin{aligned} A(x) \cdot B(x) &= (4 \cdot 8)x^6 + (4 \cdot 7 + 3 \cdot 8)x^5 + (4 \cdot 6 + 3 \cdot 7 + 2 \cdot 8)x^4 \\ &\quad + (4 \cdot 5 + 3 \cdot 6 + 2 \cdot 7 + 1 \cdot 8)x^3 + (3 \cdot 5 + 2 \cdot 6 + 1 \cdot 7)x^2 + (2 \cdot 5 + 1 \cdot 6)x + 1 \cdot 5 \\ &= 32x^6 + 52x^5 + 61x^4 + 60x^3 + 34x^2 + 16x + 5 \end{aligned}$$

である。

## 1 多項式の表現

高速化のために、以下の事実を利用する。ここでは、係数は実数または複素数であると考ええる。

**事実 1.1.** 次数高々  $d$  の多項式は、任意の相異なる  $d+1$  点における値により一意に定まる。

証明.  $A(x)$  と  $B(x)$  を次数高々  $d$  の多項式とし、相異なる  $d+1$  点  $x_0, \dots, x_d$  に対して

$$A(x_i) = B(x_i) \quad (i = 0, \dots, d)$$

が成り立つとする。このとき、差

$$H(x) = A(x) - B(x)$$

は次数高々  $d$  の多項式であり、

$$H(x_i) = 0 \quad (i = 0, \dots, d)$$

を満たす。つまり、 $H(x)$  は相異なる  $d+1$  個の根をもつ。しかし、0 でない次数高々  $d$  の多項式は高々  $d$  個の根しかもたない（以下の補題）。よって  $H(x)$  は零多項式であり、 $A(x) = B(x)$  である。したがって、次数高々  $d$  の多項式は、任意の相異なる  $d+1$  点での値により一意に定まる。□

**補題 1.2.** 0 でない次数高々  $d$  の多項式は、相異なる根を高々  $d$  個しかもたない。

証明.  $P(\alpha) = 0$  なら、 $P(x)$  を  $x - \alpha$  で割った剰余は  $P(\alpha) = 0$  である。よって  $P(x)$  は  $x - \alpha$  で割り切れる。したがって、相異なる根  $\alpha_1, \dots, \alpha_r$  をもつなら、 $(x - \alpha_1) \cdots (x - \alpha_r)$  が  $P(x)$  を割り切る。よって

$$r \leq \deg P$$

である。いま  $\deg P \leq d$  であるから、 $r \leq d$  が導かれる。□

この事実より、多項式  $A(x) = a_0 + a_1x + \cdots + a_dx^d$  は、

1. 係数  $a_0, a_1, \dots, a_d$
2. 値  $A(x_0), A(x_1), \dots, A(x_d)$

のいずれかで定めることができる。ここで、 $x_0, \dots, x_d$  は相異なる点である。

多項式  $A(x)$  と  $B(x)$  が、同じ  $n$  個の点  $x_0, \dots, x_{n-1}$  における値表現で与えられていれば、それらの積  $C(x) = A(x)B(x)$  の値表現は、各点  $x_k$  において

$$C(x_k) = A(x_k) \cdot B(x_k)$$

と計算すればよく、計算量は  $O(n)$  で済む。ただし、 $C(x)$  の次数は高々  $2d$  であるため、係数表現として一意に復元するには  $n \geq 2d + 1$  個の点が必要である。

このとき、係数表現の値表現への変換（**多項式評価** (*polynomial evaluation*））と、その逆の、値表現の係数表現への変換（**多項式補間** (*polynomial interpolation*））を高速化する方法が考えられる。以下がその手順である。

■**多項式の掛け算** 入力：多項式  $A(x)$  と  $B(x)$  の係数。出力：多項式  $C(x) = A(x) \cdot B(x)$  の係数。

1. 選択：  $n \geq 2d + 1$  に対し、相異なる点  $x_0, x_1, \dots, x_{n-1}$  を選ぶ。
2. 評価：  $A(x_0), \dots, A(x_{n-1})$  と  $B(x_0), \dots, B(x_{n-1})$  を計算する。
3. 掛け算：  $C(x_k) = A(x_k) \cdot B(x_k)$  を  $k = 0, 1, \dots, n - 1$  に対し計算する。
4. 補間：  $C(x) = c_0 + c_1x + \cdots + c_{2d}x^{2d}$  を求める。

選択と掛け算は計算量  $O(n)$  でできる。評価について、 $d$  次多項式の任意の一点の評価は計算量  $O(d)$  でできるため（ホーナー法）、 $n$  点を求めるのは計算量  $O(nd) = O(n^2)$  で可能である。高速フーリエ変換は、 $x_0, \dots, x_{n-1}$  の選び方を工夫することで、これを計算量  $O(n \log_2 n)$  で実現する方法である（図 1 参照）。

■**ホーナー法** 多項式  $A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_dx^d$  に対し、点  $z$  における値  $A(z)$  を求めたい。各幂  $z^i$  を別々に計算してから  $a_iz^i$  を足し合わせると、幂乗の計算に多くの掛け算が必要になる。一方、 $z, z^2, z^3, \dots$  を順に再利用すれば、 $A(z)$  は  $O(d)$  回の演算で計算できる。

ここでは、同じく  $O(d)$  回の演算で多項式を評価する標準的な方法として、ホーナー法 (Horner's method) を紹介する。ホーナー法は、

$$A(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{d-1} + xa_d) \cdots))$$

という括弧のくくり方にもとづく。具体的には、次の手続きで  $A(z)$  を計算する。

1.  $a = a_d$  とする。
2.  $i = d - 1, d - 2, \dots, 0$  について、 $a = a_i + za$  と更新する。
3.  $a$  を出力する。この値が  $A(z)$  である。

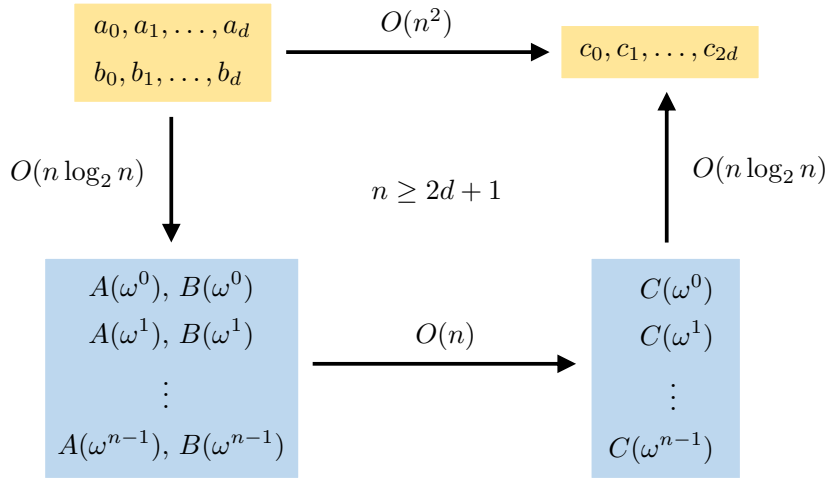


図1 高速フーリエ変換による多項式の掛け算

この方法では、各段階で掛け算と足し算を1回ずつ行うため、掛け算と足し算はともに  $O(d)$  回で済む。また、中間的に  $z^i$  を明示的に保存する必要がなく、実装も単純である。

**例 1.3.**  $A(x) = 4x^3 + 3x^2 + 2x + 1$  に対し、ホーナー法で  $A(10)$  を求めると、

$$a = 4 \longrightarrow a = 3 + 10 \times 4 = 43 \longrightarrow a = 2 + 10 \times 43 = 432 \longrightarrow a = 1 + 10 \times 432 = 4321$$

と計算を行う。

## 2 分割統治法による多項式評価

簡単のため、 $n$  は偶数とする。次数高々  $n-1$  の多項式  $A(x)$  の値  $A(x_i)$  と  $A(-x_i)$  を比べると、 $x_i$  の冪乗が偶数の項は一致し、奇数の項は符号が反転していることから、

$$\begin{aligned} A(x_i) &= a_0 + a_1x_i + a_2x_i^2 + a_3x_i^3 + \cdots + a_{n-1}x_i^{n-1} \\ &= (a_0 + a_2x_i^2 + \cdots) + (a_1x_i + a_3x_i^3 + \cdots) \\ &= (a_0 + a_2x_i^2 + \cdots) + x_i(a_1 + a_3x_i^2 + \cdots) \\ &= A_e(x_i^2) + x_iA_o(x_i^2) \\ A(-x_i) &= a_0 - a_1x_i + a_2x_i^2 - a_3x_i^3 + \cdots - a_{n-1}x_i^{n-1} \\ &= (a_0 + a_2x_i^2 + \cdots) - (a_1x_i + a_3x_i^3 + \cdots) \\ &= (a_0 + a_2x_i^2 + \cdots) - x_i(a_1 + a_3x_i^2 + \cdots) \\ &= A_e(x_i^2) - x_iA_o(x_i^2) \end{aligned}$$

のように表すことができる。ここで

$$A_e(x) = a_0 + a_2x + a_4x^2 + \cdots, \quad A_o(x) = a_1 + a_3x + a_5x^2 + \cdots$$

である。この多項式  $A_e, A_o$  は、

$$A(x) = A_e(x^2) + xA_o(x^2)$$

を満たすものであり、 $A_e(x), A_o(x)$  の次数は  $n/2 - 1$  以下である。つまり、 $A(x_i)$  と  $A(-x_i)$  の評価のために、 $A_e(x_i^2)$  と  $A_o(x_i^2)$  を用いており、次数が半分の多項式評価に帰着させる分割統治法と見ることがで

きそうである。この考察にもとづき、評価する点の選び方を、

$$\pm x_0, \pm x_1, \dots, \pm x_{n/2-1}$$

とすることが考えられる。こうすれば、異なる 2 点の評価値  $A(x_i)$  と  $A(-x_i)$  の計算のために、次数半分の多項式の評価値  $A_e(x_i^2)$  と  $A_o(x_i^2)$  を求めればよい。同様の方法で再帰的に分割できれば、 $n$  点で評価する計算量は

$$T(n) = 2T(n/2) + O(n)$$

を満たし、 $T(n) = O(n \log_2 n)$  が達成できる。

しかし、そう簡単にはいかない。 $A_e$  や  $A_o$  を同じ方法で再帰的に評価するには、評価点  $x_i^2$  たちも正負のペアになっていてほしい。すなわち、ある評価点  $x_i^2$  に対して、 $-x_i^2$  も評価点として含まれていてほしい。しかし、二乗した値が負 ( $-x_i^2$ ) であるような実数は存在しない。

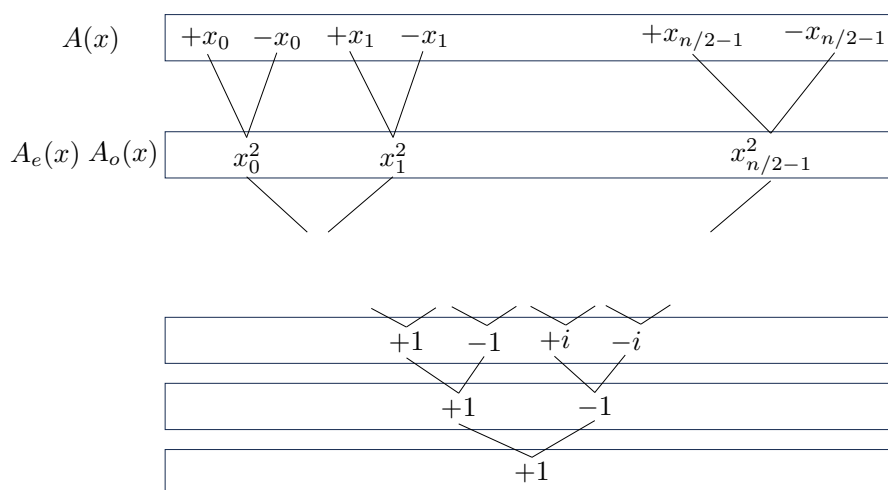


図 2 多項式の評価点

ただし、複素数 (complex number) まで考えれば、二乗して負になる数も存在する。例えば  $i^2 = -1$  である。そこで、評価点を複素数から選ぶことを考える。

再帰を最後まで続けるには、評価点の集合が次の性質をもつと都合がよい。すなわち、点たちは正負のペアに分かれ、さらに二乗すると、半分の個数の評価点集合になる、という性質である。この性質を満たす自然な候補が、1 の  $n$  乗根である。 $n$  を偶数とし、

$$\omega = e^{2\pi i/n} = \cos\left(\frac{2\pi}{n}\right) + i \sin\left(\frac{2\pi}{n}\right)$$

とおく。この  $\omega$  は 1 の原始  $n$  乗根であり、方程式  $z^n = 1$  の解全体は

$$\{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

である。これらを 1 の  $n$  乗根 (the  $n$ th roots of unity) と呼ぶ。

特に  $n$  が偶数のとき、1 の  $n$  乗根は以下の性質を満たす。

1. 正負のペアをもつ：

$$\omega^{j+n/2} = -\omega^j.$$

2. 二乗すると 1 の  $n/2$  乗根になる：

$$(\omega^j)^2 = (\omega^2)^j,$$

ここで  $\omega^2 = e^{2\pi i/(n/2)}$  であるため、 $\omega^2$  は 1 の原始  $n/2$  乗根である。

$n$  が 2 の冪乗であれば、この性質を再帰的に使うことができる。つまり、再帰の深さが  $k$  のとき、評価点は 1 の  $(n/2^k)$  乗根になり、最後は 1 点の評価に到達する。

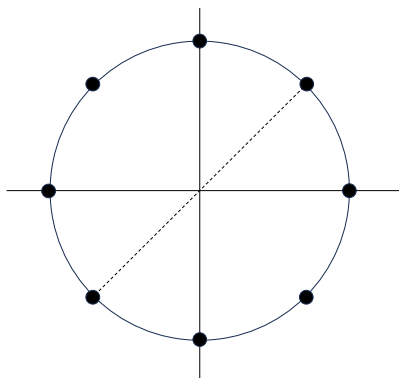


図3 1 の 8 乗根

例えば  $n = 8$  のとき、評価点は

$$1, \omega, \omega^2, \dots, \omega^7$$

である。この集合は正負のペア

$$(\omega^0, \omega^4), (\omega^1, \omega^5), (\omega^2, \omega^6), (\omega^3, \omega^7)$$

に分かれる。また、それらを二乗すると

$$1, \omega^2, \omega^4, \omega^6$$

となり、これは 1 の 4 乗根である。

これで高速フーリエ変換が実現できた。以下がそのプログラムであるが、不完全な部分が多く残っている。

```

1  /*****
2  高速フーリエ変換
3  入力： 次数  $n-1$  以下の多項式  $A(x)$  の係数表現 ( $n$ は2の冪乗), 1の $n$ 乗根  $w$ 
4  出力：  $A(x)$  の値表現  $A(w^0), \dots, A(w^{n-1})$ 
5  *****/
6  int FFT ( A, w, n ) {
7      if ( w == 1 ) return A(1);
8       $A(x)$  を  $Ae(x^2) + xAo(x^2)$  の形で表す
9      FFT( Ae,  $w^2$  ) を呼び出して  $Ae((w^2)^0), \dots, Ae((w^2)^{n/2-1})$  を得る
10     FFT( Ao,  $w^2$  ) を呼び出して  $Ao((w^2)^0), \dots, Ao((w^2)^{n/2-1})$  を得る
11     for ( j = 0; j < n/2; j++ ) {
12          $A(w^j) = Ae((w^2)^j) + w^j Ao((w^2)^j)$  を計算
13          $A(w^{j+n/2}) = Ae((w^2)^j) - w^j Ao((w^2)^j)$  を計算
14     }
15 }
```

計算としては、 $A(x) = \sum_{k=0}^{n-1} a_k x^k$  に対し、各  $j = 0, 1, \dots, n-1$  について、

$$A(\omega^j) = \sum_{k=0}^{n-1} a_k \omega^{jk} = \sum_{k=0}^{n-1} a_k \omega^{jk \bmod n}$$

を求めることである。最後の等式は、 $\omega^n = 1$  より成り立つ。この計算は、**離散フーリエ変換** (*discrete Fourier transform*) と呼ばれる。

**クイズ 2.1.** 多項式  $A(x), B(x)$  が、共通の相異なる  $n$  点  $x_0, \dots, x_{n-1}$  での値表現で与えられているとする。このとき、積  $C(x) = A(x)B(x)$  の値表現について正しいものをすべて選べ。

1.  $C(x_k) = A(x_k)B(x_k)$  で求められる。
2.  $C(x_k)$  を求めるためには、まず  $A(x)$  と  $B(x)$  を係数表現に戻す必要がある。
3. 各点での掛け算だけなら計算量は  $O(n)$  である。
4.  $C(x)$  を一意に定めるには、少なくとも  $\deg C + 1$  個の点での値が必要である。

**クイズ 2.2.** 多項式

$$A(x) = 3x^5 + 2x^4 + x^3 + 4x + 1$$

を

$$A(x) = A_e(x^2) + xA_o(x^2)$$

の形に書くとき、 $A_e(x)$  と  $A_o(x)$  として正しいものを選べ。

1.  $A_e(x) = 2x^2 + 1, \quad A_o(x) = 3x^2 + x + 4$
2.  $A_e(x) = 3x^2 + x + 4, \quad A_o(x) = 2x^2 + 1$
3.  $A_e(x) = 2x^4 + 1, \quad A_o(x) = 3x^4 + x^2 + 4$
4.  $A_e(x) = 1, \quad A_o(x) = 3x^5 + 2x^4 + x^3 + 4$

**クイズ 2.3.** 分割統治法による多項式評価について、正しいものをすべて選べ。

1. 分割統治法による多項式評価は、評価点の選び方によらず計算量  $O(n \log n)$  で実現できる。
2. 評価点として 1 の  $n$  乗根を選ぶことで、計算量  $O(n \log n)$  で実現できる。
3. 分割統治法による多項式評価では、 $A(x)$  の係数表現から任意の  $n$  点での値を計算量  $O(n)$  で実現できる。
4. 分割統治法による多項式評価の再帰的な計算量は  $T(n) = 2T(n/2) + O(n)$  である。

**クイズ 2.4.** 離散フーリエ変換 (DFT) および高速フーリエ変換 (FFT) に関して、正しいものをすべて選べ。

1. DFT は、 $n$  個の複素数  $a_0, \dots, a_{n-1}$  から、 $n$  個の複素数  $y_0, \dots, y_{n-1}$  を計算する変換である。
2. DFT の計算量は一般に  $O(n^2)$  であるが、FFT を用いると計算量  $O(n \log n)$  で実現できる。
3. FFT を用いれば、任意の  $n$  個の評価点で多項式の値を計算量  $O(n \log n)$  で実現できる。
4. 逆 FFT を用いれば、1 の  $n$  乗根での値表現から係数表現を計算量  $O(n \log n)$  で求めることができる。

### 3 多項式補間

高速フーリエ変換により、多項式  $A$  の係数表現  $(a_0, \dots, a_{n-1})$  を、1 の  $n$  乗根における値表現

$$A(1), A(\omega), \dots, A(\omega^{n-1})$$

へ、計算量  $O(n \log_2 n)$  で変換できた。関数 FFT として書けば、

$$\langle A(1), A(\omega), \dots, A(\omega^{n-1}) \rangle = \text{FFT}(\langle a_0, \dots, a_{n-1} \rangle, \omega)$$

である。

残る問題は、多項式補間で、値表現を係数表現に変換することである。これは、なんと、

$$\langle a_0, \dots, a_{n-1} \rangle = \frac{1}{n} \text{FFT}(\langle A(1), \dots, A(\omega^{n-1}) \rangle, \omega^{-1})$$

で計算できることがわかる。つまり、同じ関数 FFT を用いて計算できる。これが成り立つ理由は、線形代数で理解できる。

多項式の係数表現と値表現について、これらの変換は線形変換であることがわかる。

$$\begin{bmatrix} A(x_0) \\ A(x_1) \\ \vdots \\ A(x_{n-1}) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ & & \vdots & & \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

ここで登場する  $n \times n$  行列は *Vandermonde 行列* と呼ばれる。これを  $V_n$  と表すことにする。

**事実 3.1.**  $x_0, \dots, x_{n-1}$  が相異なるとき、Vandermonde 行列  $V_n$  は可逆 (invertible) である。

この事実は、Vandermonde 行列の行列式が

$$\prod_{0 \leq j < k \leq n-1} (x_k - x_j)$$

で与えられることからわかる。これより、 $V_n^{-1}$  が存在する。そして、多項式評価は  $V_n$  の掛け算、多項式補間は  $V_n^{-1}$  の掛け算であることがわかる。

高速フーリエ変換では、以下の  $n \times n$  行列

$$V_n(\omega) = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^k & \cdots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2k} & \cdots & \omega^{2(n-1)} \\ & & \vdots & & & & \\ 1 & \omega^j & \omega^{2j} & \cdots & \omega^{jk} & \cdots & \omega^{j(n-1)} \\ & & \vdots & & & & \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \cdots & \omega^{(n-1)k} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix}$$

を、係数表現に掛けることで、値表現にしていた。  $V_n(\omega)$  の  $(j, k)$  成分は、 $(j, k) = (0, 0)$  から数えたとき、 $\omega^{jk}$  である。標準基底  $e_k$  ( $k$  番目の要素だけが 1 で他はすべて 0) に対して  $V_n(\omega)$  を掛けると、

$$V_n(\omega) e_k = \begin{bmatrix} 1 & 1 & \cdots & 1 & \cdots & 1 \\ 1 & \omega & \cdots & \omega^k & \cdots & \omega^{n-1} \\ & & \vdots & & & \\ 1 & \omega^j & \cdots & \omega^{jk} & \cdots & \omega^{(n-1)j} \\ & & \vdots & & & \\ 1 & \omega^{n-1} & \cdots & \omega^{(n-1)k} & \cdots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ \omega^k \\ \vdots \\ \omega^{jk} \\ \vdots \\ \omega^{(n-1)k} \end{bmatrix}$$

のように、 $V_n(\omega)$  の第  $k$  列目が現れる。さらに、 $V_n(\omega)$  の各列は互いに直交する (orthogonal) ことがわかる。つまり、 $V_n(\omega)$  を掛けることは、標準基底を  $V_n(\omega)$  の列 (フーリエ基底と呼ぶ) に変換する操作である。行列  $V_n(\omega)$  による線形変換は離散フーリエ変換である。

したがって、 $V_n(\omega)$  の逆行列を掛けることは、フーリエ基底を標準基底に戻す操作であり、**逆フーリエ変換** (inverse Fourier transform) と呼ばれる。

列ベクトル  $\mathbf{u} = (u_0, \dots, u_{n-1})^T$ ,  $\mathbf{v} = (v_0, \dots, v_{n-1})^T \in \mathbb{C}^n$  に対し、内積 (inner product) を

$$\langle \mathbf{u}, \mathbf{v} \rangle := u_0^* v_0 + u_1^* v_1 + \dots + u_{n-1}^* v_{n-1}$$

で定める。ここで、 $w^*$  は複素数  $w$  の複素共役 (complex conjugate) を表す。例えば、 $w = ae^{i\theta}$  に対して  $w^* = ae^{-i\theta}$  である。また、行列  $M$  に対して、 $M^*$  は  $M$  の共役転置を表す。

**補題 3.2.**  $V_n(\omega)$  の  $j$  列目を

$$\mathbf{v}_{(j)} := (1, \omega^j, \omega^{2j}, \dots, \omega^{(n-1)j})^T$$

と表すとき ( $j = 0, \dots, n-1$ ),

$$\langle \mathbf{v}_{(j)}, \mathbf{v}_{(k)} \rangle = \begin{cases} 0 & j \neq k, \\ n & j = k \end{cases}$$

である。つまり、 $V_n(\omega)$  の相異なる列は直交する。

証明.  $j \neq k$  のとき、

$$\begin{aligned} \langle \mathbf{v}_{(j)}, \mathbf{v}_{(k)} \rangle &= 1 + \omega^{k-j} + \omega^{2(k-j)} + \dots + \omega^{(n-1)(k-j)} \\ &= \frac{1 - \omega^{n(k-j)}}{1 - \omega^{k-j}} = 0. \end{aligned}$$

最後の等号は、 $\omega^n = 1$  より  $\omega^{n(k-j)} = 1$  であり、また  $j \neq k$  なので  $\omega^{k-j} \neq 1$  であることから従う。 $j = k$  のときは、各項が 1 なので、

$$\langle \mathbf{v}_{(j)}, \mathbf{v}_{(j)} \rangle = n$$

である。 □

補題より、 $V_n(\omega)$  の列ベクトルは互いに直交し、各列のノルムの二乗は  $n$  である。したがって、

$$V_n(\omega)^* V_n(\omega) = nI$$

が成り立つ。これより、

$$V_n(\omega)^{-1} = \frac{1}{n} V_n(\omega)^*$$

である。

また、 $V_n(\omega)^*$  の  $(j, k)$  成分は  $\omega^{-jk}$  である。したがって、

$$V_n(\omega)^* = V_n(\omega^{-1})$$

であり、

$$V_n(\omega)^{-1} = \frac{1}{n} V_n(\omega^{-1})$$

が導かれる。つまり、逆フーリエ変換は、 $\omega^{-1}$  を用いた FFT を計算し、最後に各成分を  $n$  で割ることで実現できる。ここで  $\omega^{-1}$  も 1 の原始  $n$  乗根であるため、 $V_n(\omega^{-1})$  も FFT で高速に計算できる。

**■ラグランジュの補間公式** 多項式  $p(x)$  から任意の点  $z$  における値  $p(z)$  を求める計算は、ホーナー法により計算量  $O(n)$  ででき、 $n$  点あれば合計計算量  $O(n^2)$  でできた。FFT はそれを特定の  $n$  点に対して  $O(n \log_2 n)$  で計算する手法であった。

一方、任意の異なる  $n$  点  $x_0, \dots, x_{n-1}$  における値  $A(x_0), \dots, A(x_{n-1})$  から多項式  $A(x)$  を求める問題も考えられる。これは Vandermonde 行列を用いれば連立一次方程式として表せるが、一般の方法で連立一次方程式を解くと、素朴には  $O(n^3)$  の計算量がかかる。

任意の  $n$  評価点からの多項式補間を計算量  $O(n^2)$  で実現する方法としてラグランジュの補間公式 (Lagrange interpolation formula) が知られている.  $A(x_k) = y_k$  ( $k = 0, \dots, n-1$ ) に対し,

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}$$

で与えられる. この多項式の次数は高々  $n-1$  であり, 各  $k$  について  $A(x_k) = y_k$  であることが確認できる. また, この多項式の係数表現は計算量  $O(n^2)$  で求められる (演習問題 6.6 を参照せよ).

**例 3.3.**  $A(0) = 1, A(1) = 10, A(2) = 49, A(3) = 142$  からラグランジュ補間により 3 次多項式  $A(x)$  を求めると,

$$\begin{aligned} A(x) &= 1 \cdot \frac{(x-1)(x-2)(x-3)}{(0-1)(0-2)(0-3)} + 10 \cdot \frac{(x-0)(x-2)(x-3)}{(1-0)(1-2)(1-3)} + 49 \cdot \frac{(x-0)(x-1)(x-3)}{(2-0)(2-1)(2-3)} \\ &\quad + 142 \cdot \frac{(x-0)(x-1)(x-2)}{(3-0)(3-1)(3-2)} \\ &= \frac{(x-1)(x-2)(x-3)}{-6} + 10 \cdot \frac{x(x-2)(x-3)}{2} + 49 \cdot \frac{x(x-1)(x-3)}{-2} + 142 \cdot \frac{x(x-1)(x-2)}{6} \\ &= 4x^3 + 3x^2 + 2x + 1 \end{aligned}$$

と求まる.

**クイズ 3.1.** 多項式補間に関して, 正しいものをすべて選べ.

1. 次数高々  $d$  の多項式は, 任意の異なる  $d+1$  点での値によって一意に定まる.
2. 多項式補間は, 値表現から係数表現を復元する操作である.
3. Vandermonde 行列が可逆であれば, 値表現に対応する係数表現は一意に定まる.
4. 高速フーリエ変換を用いると, 1 の  $n$  乗根での値から係数表現を計算量  $O(n \log n)$  で求めることができる.
5. 任意の  $n$  点での値から FFT を用いて係数表現を計算量  $O(n \log n)$  で求めることができる.

## 4 高速フーリエ変換（アルゴリズムと再帰計算）

高速フーリエ変換は、列ベクトル  $\mathbf{a} = (a_0, a_1, \dots, a_{n-1})^T$  と 1 の原始  $n$  乗根  $\omega$  を入力として受けとり、 $\mathbf{a}$  に  $V_n(\omega)$  を左から掛ける。分割統治法により再帰的に計算する過程を見てみよう。

まず、係数ベクトル  $\mathbf{a}$  を偶数番目の係数と奇数番目の係数に分ける。すなわち、

$$\mathbf{a}_e = (a_0, a_2, \dots, a_{n-2})^T, \quad \mathbf{a}_o = (a_1, a_3, \dots, a_{n-1})^T$$

とする。これは、多項式を

$$A(x) = A_e(x^2) + xA_o(x^2)$$

と分解することに対応している。

次に、 $V_n(\omega)$  の行を  $j$  行目 ( $0 \leq j < n/2$ ) と  $j+n/2$  行目に分ける。ここで、 $\omega^{n/2} = -1$  であることに注意すると、 $\omega^{j(2k)} = (\omega^2)^{jk}$ ,  $\omega^{j(2k+1)} = \omega^j \cdot (\omega^2)^{jk}$ ,  $\omega^{(j+n/2)(2k)} = (\omega^2)^{jk}$ ,  $\omega^{(j+n/2)(2k+1)} = -\omega^j \cdot (\omega^2)^{jk}$  が成り立つ。したがって、 $V_{n/2}(\omega^2)$  に対し、 $(a_0, a_2, \dots, a_{n-2})$  を掛けたものと、 $(a_1, a_3, \dots, a_{n-1})$  を掛けたものを計算できればよいことがわかる（図 4）。

具体的に、

$$(s_0, \dots, s_{n/2-1})^T = V_{n/2}(\omega^2)\mathbf{a}_e, \quad (t_0, \dots, t_{n/2-1})^T = V_{n/2}(\omega^2)\mathbf{a}_o$$

とおく。このとき、 $0 \leq j < n/2$  について

$$r_j = s_j + \omega^j t_j, \quad r_{j+n/2} = s_j - \omega^j t_j$$

と計算すれば、

$$(r_0, \dots, r_{n-1})^T = V_n(\omega)\mathbf{a}$$

が得られる。

この関係を図式化したものが図 4 であり、これをもとにしたプログラムが図 5 である。

図 5 のアルゴリズムでは、サイズ  $n/2$  の FFT を 2 回呼び出し、最後の for ループで  $O(n)$  回の演算を行う。したがって計算量  $T(n)$  は

$$T(n) = 2T(n/2) + O(n)$$

を満たす。よって、 $T(n) = O(n \log_2 n)$  である。

理解を深めるため、高速フーリエ変換 (FFT) の再帰計算の中身を見てみよう。図 6 は、入力サイズ  $n$  に対する計算回路  $\text{FFT}_n$  の様子である。

入力  $a_0, \dots, a_{n-1}$  は偶数番号と奇数番号に分けられ、サイズ  $n/2$  の回路  $\text{FFT}_{n/2}$  にそれぞれ接続している。偶数番号の出力を  $(s_0, \dots, s_{n/2-1})$ 、奇数番号の出力を  $(t_0, \dots, t_{n/2-1})$  としたとき、出力は

$$r_j = s_j + \omega^j t_j, \quad r_{j+n/2} = s_j - \omega^j t_j$$

と計算される。また、 $\omega^{j+n/2} = -\omega^j$  であるため、

$$r_{j+n/2} = s_j + \omega^{j+n/2} t_j$$

とも書ける。ワイヤのラベル  $\omega^j, \omega^{j+n/2}$  は、その値を掛けて足し合わせることを表し、ラベルのないワイヤは値をそのまま足し合わせることを表す。

より具体的に、 $n = 4$  の場合の計算過程を表したのが図 7 である。

$$\begin{aligned}
V_n(\omega) \mathbf{a} &= \begin{bmatrix} \vdots & & \\ \dots & \omega^{jk} & \dots \\ \vdots & & \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-2} \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} \vdots & & \\ \dots & \omega^{j(2k)} & \dots & \vdots & \dots \\ \vdots & & & \vdots & \dots \end{bmatrix} \begin{bmatrix} a_0 \\ a_2 \\ \vdots \\ \frac{a_{n-2}}{a_1} \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \vdots & & \\ \dots & \omega^{j(2k)} & \dots & \vdots & \dots \\ \vdots & & & \vdots & \dots \end{bmatrix} \begin{bmatrix} a_0 \\ a_2 \\ \vdots \\ a_{n-4} \\ a_{n-2} \\ a_1 \\ a_3 \\ \vdots \\ a_{n-3} \\ a_{n-1} \end{bmatrix} \\
&= \begin{bmatrix} \boxed{V_{n/2}(\omega^2)} & \begin{bmatrix} a_0 \\ a_2 \\ \vdots \\ a_{n-4} \\ a_{n-2} \end{bmatrix} & + \omega^j & \boxed{V_{n/2}(\omega^2)} & \begin{bmatrix} a_1 \\ a_3 \\ \vdots \\ a_{n-3} \\ a_{n-1} \end{bmatrix} \\ \boxed{V_{n/2}(\omega^2)} & \begin{bmatrix} a_0 \\ a_2 \\ \vdots \\ a_{n-4} \\ a_{n-2} \end{bmatrix} & - \omega^j & \boxed{V_{n/2}(\omega^2)} & \begin{bmatrix} a_1 \\ a_3 \\ \vdots \\ a_{n-3} \\ a_{n-1} \end{bmatrix} \end{bmatrix}
\end{aligned}$$

図4 高速フーリエ変換の再帰計算の行列表現

最初に  $(a_0, a_1, a_2, a_3)$  と  $\omega$  が入力として与えられる．次の呼び出しは

$$\text{FFT}((a_0, a_2), \omega^2, 2), \quad \text{FFT}((a_1, a_3), \omega^2, 2)$$

である．さらに，前者は  $\text{FFT}((a_0), \omega^4, 1)$  と  $\text{FFT}((a_2), \omega^4, 1)$  を呼び出し，後者は  $\text{FFT}((a_1), \omega^4, 1)$  と  $\text{FFT}((a_3), \omega^4, 1)$  を呼び出す．ここで，各呼び出しの入力サイズは1であるため， $a_0, a_2, a_1, a_3$  がそのまま返される．その後，サイズ2のFFTの出力を組み合わせる計算を行う． $\text{FFT}((a_0, a_2), \omega^2, 2)$  の計算結果として

$$(a_0 + \omega^0 a_2, a_0 - \omega^0 a_2)$$

が出力され， $\text{FFT}((a_1, a_3), \omega^2, 2)$  の計算結果として

$$(a_1 + \omega^0 a_3, a_1 - \omega^0 a_3)$$

```

1  /*****
2  高速フーリエ変換
3  入力:  $a = (a_0, a_1, \dots, a_{n-1})$  ( $n$ は2の冪乗), 1の原始 $n$ 乗根  $w$ 
4  出力:  $V_n(w) a$ 
5  *****/
6  FFT ( a, w, n ) {
7      if ( n == 1 ) return a;
8      (s_0, s_1, ..., s_{n/2-1}) = FFT( (a_0, a_2, ..., a_{n-2}), w^2, n/2 )
9      (t_0, t_1, ..., t_{n/2-1}) = FFT( (a_1, a_3, ..., a_{n-1}), w^2, n/2 )
10     for ( j = 0; j < n/2; j++ ) {
11         r_j = s_j + w^j t_j
12         r_{j+n/2} = s_j - w^j t_j
13     }
14     return (r_0, r_1, ..., r_{n-1})
15 }

```

図5 高速フーリエ変換

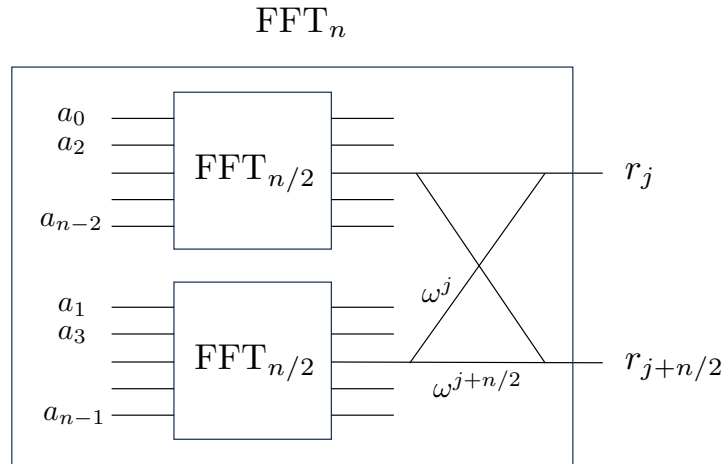


図6 入力  $a_0, \dots, a_{n-1}$  出力  $r_0, \dots, r_{n-1}$  の  $\text{FFT}_n$  回路

が出力される．最終的な  $\text{FFT}((a_0, a_1, a_2, a_3), \omega, 4)$  の出力は,

$$\begin{aligned}
 (a_0 + \omega^0 a_2) + \omega^0 (a_1 + \omega^0 a_3) &= a_0 + a_1 + a_2 + a_3 = A(\omega^0) \\
 (a_0 - \omega^0 a_2) + \omega^1 (a_1 - \omega^0 a_3) &= a_0 + a_1 \omega + a_2 \omega^2 + a_3 \omega^3 = A(\omega^1) \\
 (a_0 + \omega^0 a_2) - \omega^0 (a_1 + \omega^0 a_3) &= a_0 + a_1 \omega^2 + a_2 \omega^4 + a_3 \omega^6 = A(\omega^2) \\
 (a_0 - \omega^0 a_2) - \omega^1 (a_1 - \omega^0 a_3) &= a_0 + a_1 \omega^3 + a_2 \omega^6 + a_3 \omega^9 = A(\omega^3)
 \end{aligned}$$

と計算されることがわかる．

この計算回路を,  $n = 8$  の場合で示したのが図 8 である．この図から, 計算量だけでなく, 入力の並び替えや各入力が各出力へどのように寄与するかも読み取れる．発展的な観察として, 次の点がある．

1. 入力が  $n$  個のとき, 計算は  $\log_2 n$  段あるため, 合計の計算量は  $O(n \log_2 n)$  である．
2. 回路への入力は  $(a_0, a_4, a_2, a_6, a_1, a_5, a_3, a_7)$  という奇妙な並びに見えるが, 添字を 3 ビットの二進法表現で書き, そのビットを逆順に読んだ順に対応している．
3. 入力  $a_j$  と出力  $A(\omega^k)$  の間には一つのパスが存在し, そのたどり方は,  $k$  の二進法表現を右側から順に読み込めばよい．頂点  $a_j$  から上への分岐が 0, 下への分岐が 1 に対応している．

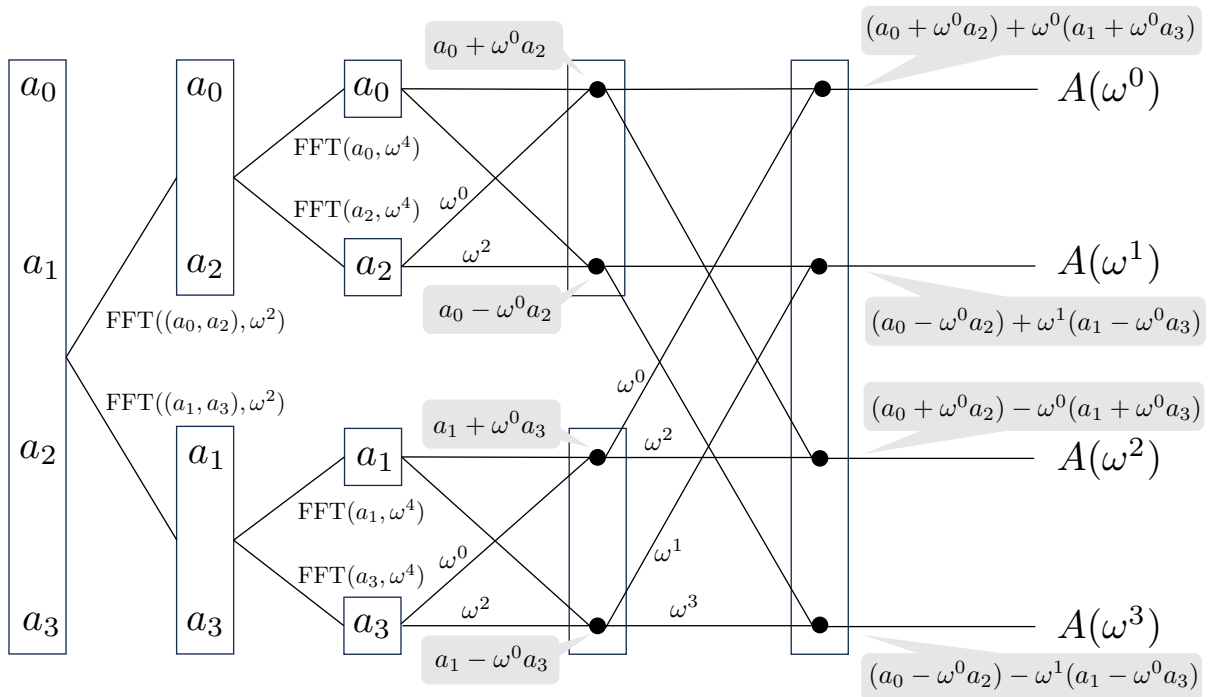


図7 高速フーリエ変換の再帰計算の過程 ( $n = 4$ )

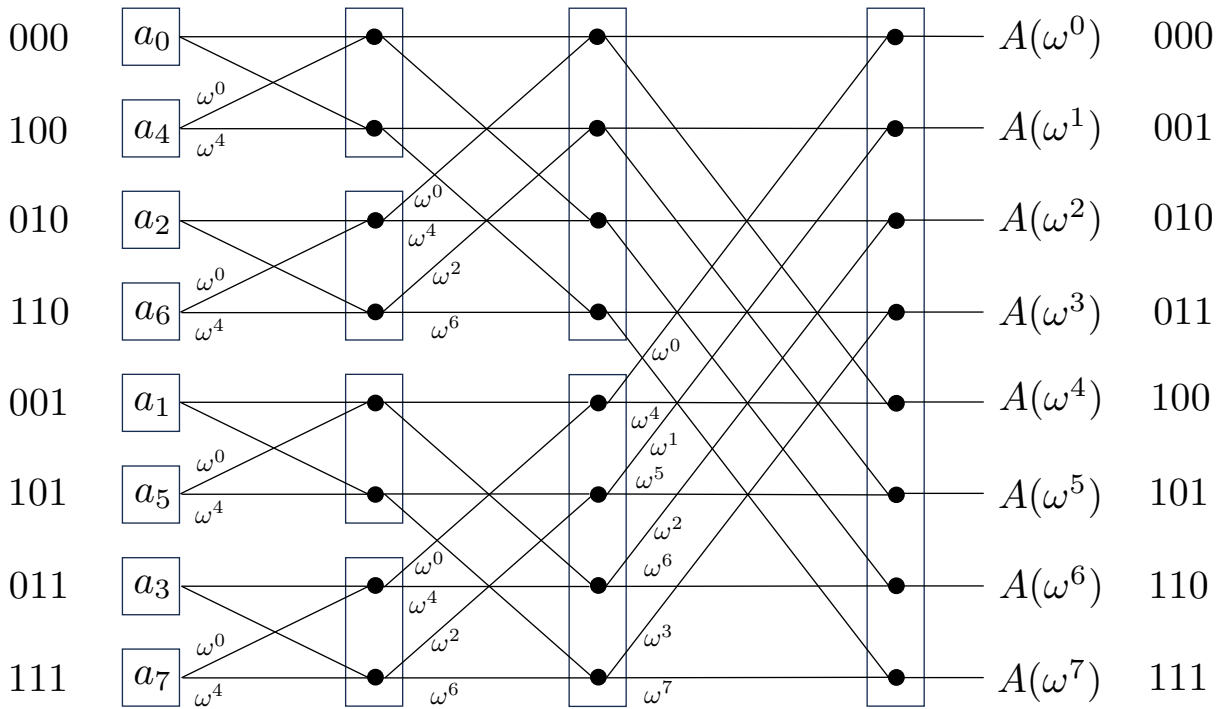


図8 高速フーリエ変換の計算回路 ( $n = 8$ )

4.  $a_j$  と  $A(\omega^k)$  の間のパスのラベル  $\omega^\ell$  の指数  $\ell$  を足し合わせると、 $jk \bmod 8$  に一致する。したがって、入力  $a_j$  の出力  $A(\omega^k)$  における寄与は  $a_j \omega^{jk}$  であり、回路が DFT を正しく計算していることがわかる。
5. 高速フーリエ変換の計算は並列化可能であり、回路として記述できる。

## 5 高速フーリエ変換の応用：整数の掛け算

高速フーリエ変換 (FFT) により、多項式の掛け算を  $O(n \log_2 n)$  ステップで計算できることを見た。ここでは、その応用として整数の掛け算を考える。

ベクトル

$$\mathbf{a} = (a_0, a_1, \dots, a_{n-1}), \quad \mathbf{b} = (b_0, b_1, \dots, b_{n-1})$$

に対し、範囲外の添字では  $a_j = 0, b_j = 0$  と定める。このとき、 $\mathbf{a}$  と  $\mathbf{b}$  の畳み込み (convolution)  $\mathbf{a} * \mathbf{b}$  を

$$(\mathbf{a} * \mathbf{b})_k = \sum_{j=0}^{n-1} a_j b_{k-j} \quad (k = 0, 1, \dots, 2n-2)$$

で定める。これは長さ  $2n-1$  のベクトルである。

多項式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i$$

の積を

$$C(x) = A(x)B(x) = \sum_{k=0}^{2n-2} c_k x^k$$

と書くと、

$$c_k = \sum_{j=0}^{n-1} a_j b_{k-j}$$

である。つまり、多項式の掛け算は、係数ベクトルの畳み込みを求めることに対応している。

離散フーリエ変換 (DFT) は、多項式  $A(x) = \sum_{i=0}^{n-1} a_i x^i$  に対応するベクトル  $\mathbf{a} = (a_0, \dots, a_{n-1})$  から、 $j = 0, 1, \dots, n-1$  について

$$y_j = \sum_{k=0}^{n-1} a_k \omega^{jk}$$

を求める変換であった。この変換を

$$\text{DFT}_n(\mathbf{a}) = (y_0, \dots, y_{n-1})$$

と書く。すでに出てきた記法を用いれば、

$$\text{DFT}_n(\mathbf{a}) = V_n(\omega)\mathbf{a}$$

である。

FFT により、 $\text{DFT}_n$  ならびに  $\text{DFT}_n^{-1}$  は  $O(n \log_2 n)$  ステップで計算できる。さらに、畳み込みは DFT を用いて次のように計算できる。

**定理 5.1** (畳み込み定理 (convolution theorem)). 長さ  $n$  のベクトル  $\mathbf{a}, \mathbf{b}$  について、 $n$  が 2 の冪乗であるとする。 $\mathbf{a}, \mathbf{b}$  の末尾に 0 を追加して長さ  $2n$  のベクトルとみなすと、

$$\text{DFT}_{2n}^{-1}(\text{DFT}_{2n}(\mathbf{a}) \cdot \text{DFT}_{2n}(\mathbf{b}))$$

の先頭  $2n-1$  成分は  $\mathbf{a} * \mathbf{b}$  に一致し、最後の成分は 0 である。ただし、右辺の  $\cdot$  は要素ごとの掛け算を表す。

ここで長さを  $2n$  にしているのは、多項式積の次数が最大で  $2n - 2$  になるためである。0 を追加して十分な長さにしておけば、DFT 上の要素ごとの積から、通常多項式積の係数列を復元できる。

$n$  ビットの整数  $a$  と  $b$  の 2 進法表現を

$$a = \sum_{i=0}^{n-1} a_i 2^i, \quad b = \sum_{i=0}^{n-1} b_i 2^i$$

とする。ただし、各  $a_i, b_i$  は 0 または 1 である。これらを係数にもつ多項式

$$A(x) = \sum_{i=0}^{n-1} a_i x^i, \quad B(x) = \sum_{i=0}^{n-1} b_i x^i$$

を考えると、

$$a = A(2), \quad b = B(2)$$

である。したがって、

$$C(x) = A(x)B(x) = \sum_{i=0}^{2n-2} c_i x^i$$

を求めれば、

$$ab = C(2) = \sum_{i=0}^{2n-2} c_i 2^i$$

である。

係数  $c_k$  は

$$c_k = \sum_{j=0}^{n-1} a_j b_{k-j}$$

で与えられる。ただし、添字が範囲外のときは  $b_{k-j} = 0$  と考える。この係数列  $(c_0, c_1, \dots, c_{2n-2})$  は、多項式積の係数列であり、そのままでは  $ab$  の 2 進法表現ではない。実際、 $c_k$  は 0 または 1 とは限らず、複数の積  $a_j b_{k-j}$  の和になっている。そこで、最下位の係数から順に桁上りを処理する。すなわち、各  $k$  について、

$$c_{k+1} \leftarrow c_{k+1} + \left\lfloor \frac{c_k}{2} \right\rfloor, \quad c_k \leftarrow c_k \bmod 2$$

を行えば、最終的に  $ab$  の 2 進法表現が得られる。

より一般的に、

$$a = \sum_{i=0}^{\ell} a_i D^i, \quad b = \sum_{i=0}^{\ell} b_i D^i$$

のように、底を  $D$  として表してもよい。例えば、 $D = 10, 2^3, 2^{16}$  などを用いることができる。この場合も、対応する多項式

$$A(x) = \sum_{i=0}^{\ell} a_i x^i, \quad B(x) = \sum_{i=0}^{\ell} b_i x^i$$

の積  $C(x) = A(x)B(x)$  を FFT で求め、最後に底  $D$  に関する桁上りを処理すれば、整数の積  $ab$  が得られる。

ただし、複素数上の DFT を用いる実装では、小数の丸め誤差や計算誤差を考慮する必要がある。誤差を避けたい場合には、素数  $p$  に対する法  $p$  での演算など、有限体上の計算を用いる方法もある。そのためには、その有限体上で 1 の原始  $n$  乗根が存在するように、法  $p$  や有限体を選ぶ必要がある。このような方法は、数論変換 (number theoretic transform, NTT) と呼ばれ、複素数を用いない FFT として理解できる。

クイズ 5.1.  $a = (1, 1, 0)$ ,  $b = (1, 0, 1)$  とする. 畳み込み  $a * b$  として正しいものを選べ.

1.  $(1, 1, 1, 1, 0)$
2.  $(1, 1, 1)$
3.  $(1, 0, 2, 0, 1)$
4.  $(1, 1, 0, 1, 0)$

## 6 高速フーリエ変換 (FFT) の演習問題

演習問題 6.1. 1 の  $n$  乗根の和を求めよ.  $n$  が奇数のとき, 1 の  $n$  乗根の積を求めよ.  $n$  が偶数のときはどうなるだろうか.

演習問題 6.2.  $p(1) = 2, p(2) = 1, p(3) = 0, p(4) = 4, p(5) = 0$  である 4 次多項式  $p$  を求め, その係数表現を与えよ.

演習問題 6.3. アダマール行列 (Hadamard matrix)  $H_0, H_1, \dots$  は以下のように定義される.

- $H_0$  は  $1 \times 1$  行列  $[1]$ .
- $k > 0$  に対し,  $H_k$  は  $2^k \times 2^k$  行列

$$\left[ \begin{array}{c|c} H_{k-1} & H_{k-1} \\ \hline H_{k-1} & -H_{k-1} \end{array} \right]$$

である.

長さ  $n = 2^k$  の列ベクトル  $v$  に対し,  $H_k v$  を  $O(n \log_2 n)$  回の演算 (足し算や掛け算) で計算できることを示せ.

演習問題 6.4. モジュロ演算におけるフーリエ変換 (FT) を考える. ここでは法 7 とする.

1. 冪乗  $\omega, \omega^2, \dots, \omega^6$  がすべて異なる  $\omega$  が存在する. そのような  $\omega$  を見つけ,  $\omega + \omega^2 + \dots + \omega^6 = 0$  を示せ. (法が素数であればそのような  $\omega$  が必ず存在する.)
2. 行列による FT を使って,  $(0, 1, 1, 1, 5, 2)$  の法 7 における変換を求めよ. つまり,  $V_6(\omega)$  を掛ければよい.

演習問題 6.5. 次数  $n$  の多項式  $A(x)$  の点  $x_0$  における値  $A(x_0)$  を求める別の方法は,  $A(x)$  を多項式  $(x - x_0)$  で割り, 次数  $n - 1$  の商多項式  $q(x)$  と剰余  $r$  を

$$A(x) = (x - x_0)q(x) + r$$

と計算することである.  $A(x_0) = r$  である.  $A(x)$  と  $x_0$  から,  $q(x)$  と  $r$  を計算量  $O(n)$  で求める方法を示せ.

演習問題 6.6. ラグランジュの補間公式を用いて計算量  $O(n^2)$  で多項式補間を行う方法を示せ. (ヒント: 多項式  $\prod_j (x - x_j)$  の係数表現を求め, 各項で分子を  $(x - x_k)$  で割る.  $n$  個ある分母は  $O(n)$  で計算できる.)

演習問題 6.7. 多項式  $A(x) = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$  に対し,  $A^{\text{rev}}(x) = a_{n-1} + a_{n-2}x + \dots + a_0x^{n-1}$  と定める.  $A(x)$  の値表現から  $A^{\text{rev}}(x)$  の値表現を求める方法を与えよ. ただし, 評価する点は 0 でないと仮定する.

**演習問題 6.8.** 0 から  $10n$  までの  $n$  個の整数からなる集合  $A, B$  に対し, その直和 (Cartesian sum) を

$$C := \{a + b \mid a \in A, b \in B\}$$

と定める.  $C$  の各要素が  $A$  と  $B$  の要素の和として何回登場するかを計算量  $O(n \log n)$  で求めよ. (ヒント:  $A$  ならびに  $B$  を次数  $10n$  以下の多項式として表現せよ.)

**演習問題 6.9.**  $\omega_k := e^{2\pi i/k}$  とする.

1. 任意の整数  $n > 0, k \geq 0, d > 0$  に対し,  $\omega_{dn}^{dk} = \omega_n^k$  であることを示せ.
2. 任意の偶数  $n > 0$  に対し,  $\omega_n^{n/2} = \omega_2 = -1$  であることを示せ.
3. 任意の偶数  $n > 0$  に対し, 1 の  $n$  乗根の平方は, 1 の  $n/2$  乗根であることを示せ.
4. 任意の整数  $n \geq 1$  と  $n$  で割り切れない非負整数  $k$  に対し,

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$$

であることを示せ.

**演習問題 6.10.**  $(1, 0, 0, 0)$  の FFT を求めよ. また,  $(1, 0, 0, 0)$  は何の FFT であるか. 同様に,  $(1, 0, 1, -1)$  に対しても求めよ.

**演習問題 6.11.** FFT を用いて多項式  $x + 1$  と  $x^2 + 1$  の積を計算したい. 2 の冪乗を適切に選び, この 2 つの多項式の FFT を求め, その結果に対し要素毎に積を取り, 逆フーリエ変換を求めることで最終的な答えを導け. 同様に,  $2x^2 + x + 1$  と  $3x + 2$  に対して計算せよ.

## クイズの答え

クイズ 2.1 正解：1, 3, 4. 値表現では、各点で  $C(x_k) = A(x_k)B(x_k)$  を計算すればよい。ただし、積の多項式を一意に定めるには、次数に応じた十分な個数の評価点が必要である。

クイズ 2.2 正解：1. 偶数次の項から  $A_e(x) = 2x^2 + 1$ , 奇数次の項から  $A_o(x) = 3x^2 + x + 4$  を得る。

クイズ 2.3 正解：2, 4. 評価点として 1 の  $n$  乗根を用いることで、偶奇分解を再帰的に使える。再帰式は  $T(n) = 2T(n/2) + O(n)$  である。

クイズ 2.4 正解：1, 2, 4. FFT は DFT を高速に計算するアルゴリズムであり、任意の評価点に対する高速評価法ではない。

クイズ 3.1 正解：1, 2, 3, 4. 任意の  $n$  点での補間は一般には FFT では計算できない。FFT で高速に補間できるのは、評価点が 1 の  $n$  乗根である場合である。

クイズ 5.1 正解：1.

$$(1+x)(1+x^2) = 1+x+x^2+x^3$$

なので、係数列は  $(1, 1, 1, 1, 0)$  である。